



JMPM

Guia Completo: RPO Bytecode Injector

DATA

30/09/2026

ARQUIVO

RPO-BYTECODE-INJECTOR-COMPLETE-GUIDE.md



Sumário

- 1. Visão Geral
- 2. Arquitetura do Sistema
- 3. Formato RPO
- 4. Criptografia e Cifras
- 5. Parser RPO
- 6. Parser APO
- 7. Heurística de Cipher
- 8. Injeção de Bytecode
- 9. ReplaceAPO Dinâmico
- 0. Recriptografia
- .1. CLI e Uso
- .2. Testes
- .3. Limitações
- .4. Próximos Passos
- .5. Referências

1. Visão Geral

1.1 Objetivo

O **RPO Bytecode Injector** é um sistema completo para injetar bytecode compilado em arquivos RPO (Repositório de Programas Objeto) do TOTVS Protheus, bypassando limitações do compilador oficial.

1.2 Problema Resolvido

O compilador oficial do Protheus (`appsrvlinux -compile`) possui limitações:

- Não permite injeção direta de bytecode pré-compilado
- Não suporta hot-reload de funções sem recompilação completa
- Pode crashar com `tAssertException` em certos cenários

1.3 Solução

O sistema implementa:

- 1. **Parsing completo** do formato RPO
- 2. **Decodificação** usando captura ao vivo de chaves de criptografia
- 3. **Extração** de registros APO (Advanced Program Object)
- 4. **Substituição** de bytecode com suporte a mudanças de tamanho
- 5. **Recriptografia** e reconstrução do RPO

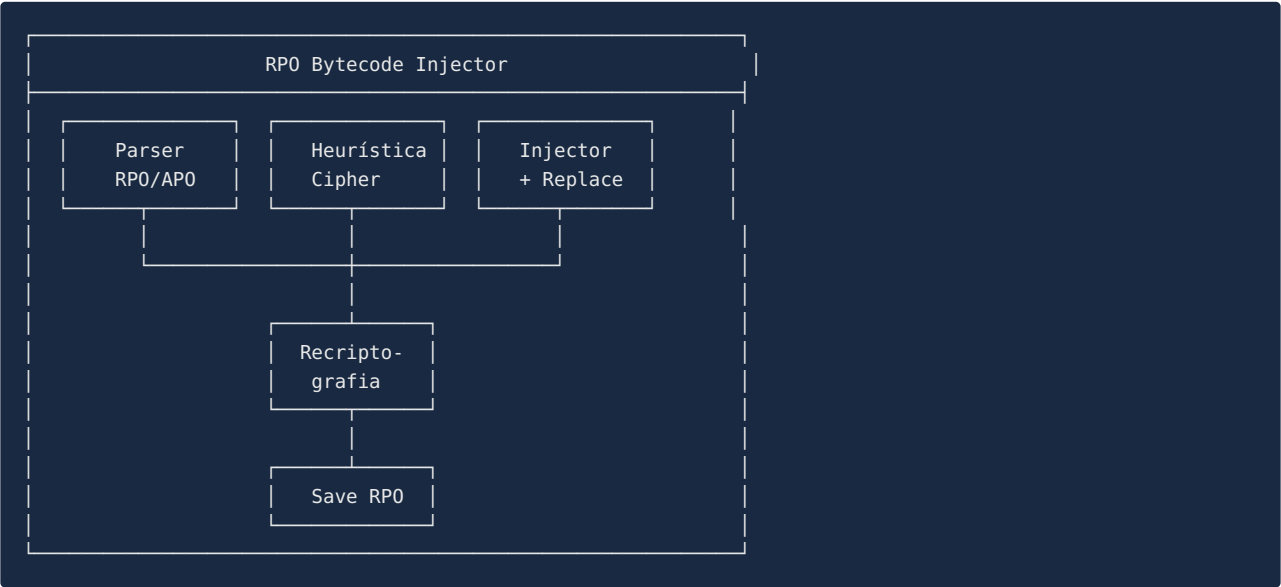
1.4 Status

Componente	Status
Parser RPO	✔ Funcional
Parser APO	✔ Funcional
Heurística Cipher	✔ 73%+

ReplaceAPO	✓ Dinâmico
Recryptografia	✓ Funcional
Testes	✓ 17/17 passing

2. Arquitetura do Sistema

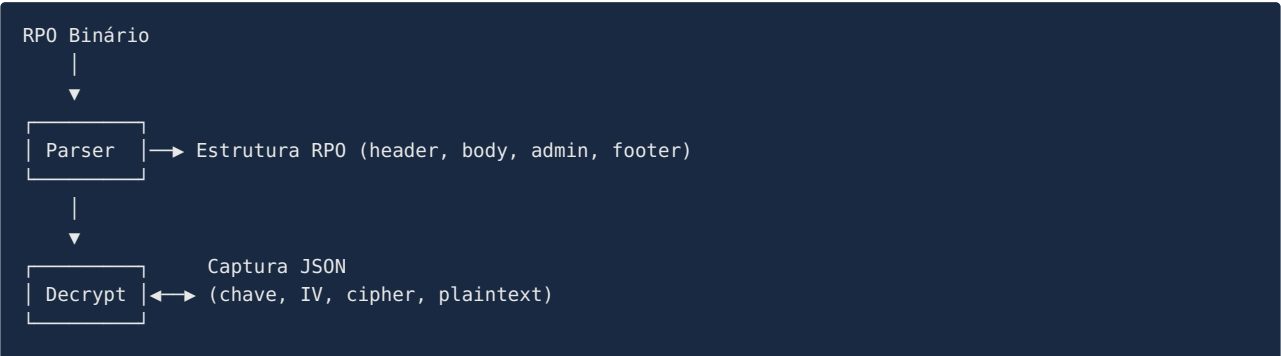
2.1 Visão Geral

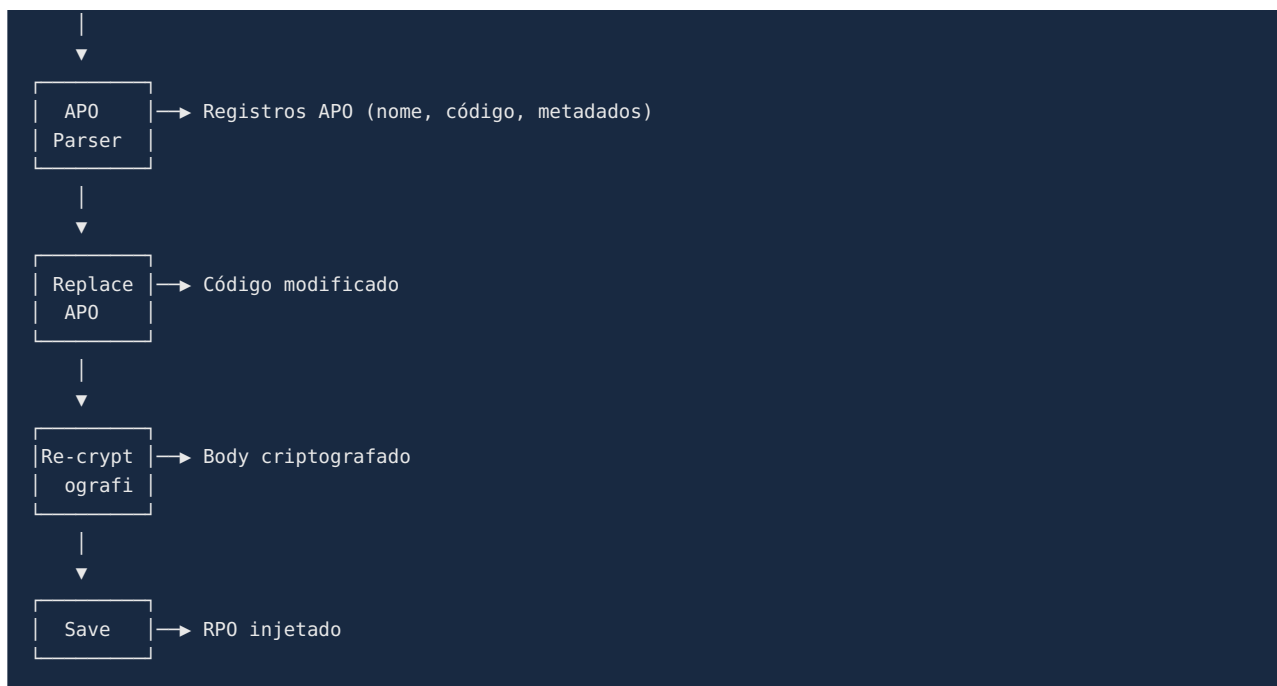


2.2 Componentes

Componente	Arquivo	Função
Parser RPO	<code>pkg/rpo/rpo.go</code>	Parsear estrutura do container RPO
Parser APO	<code>pkg/rpo/injector/injector.go</code>	Extração de registros APO
Heurística	<code>pkg/rpo/cipher_heuristic.go</code>	Identificação de ciphers
Cipher Dispatch	<code>pkg/rpo/cipher_dispatch.go</code>	Encrypt/Decrypt segments
Capture Parser	<code>pkg/rpo/capture_parser.go</code>	Parsear captura de chaves
Injector	<code>pkg/rpo/injector/injector.go</code>	Injeção e substituição
CLI	<code>cmd/advplc/cmd_rpo_inject.go</code>	Interface linha de comando

2.3 Fluxo de Dados

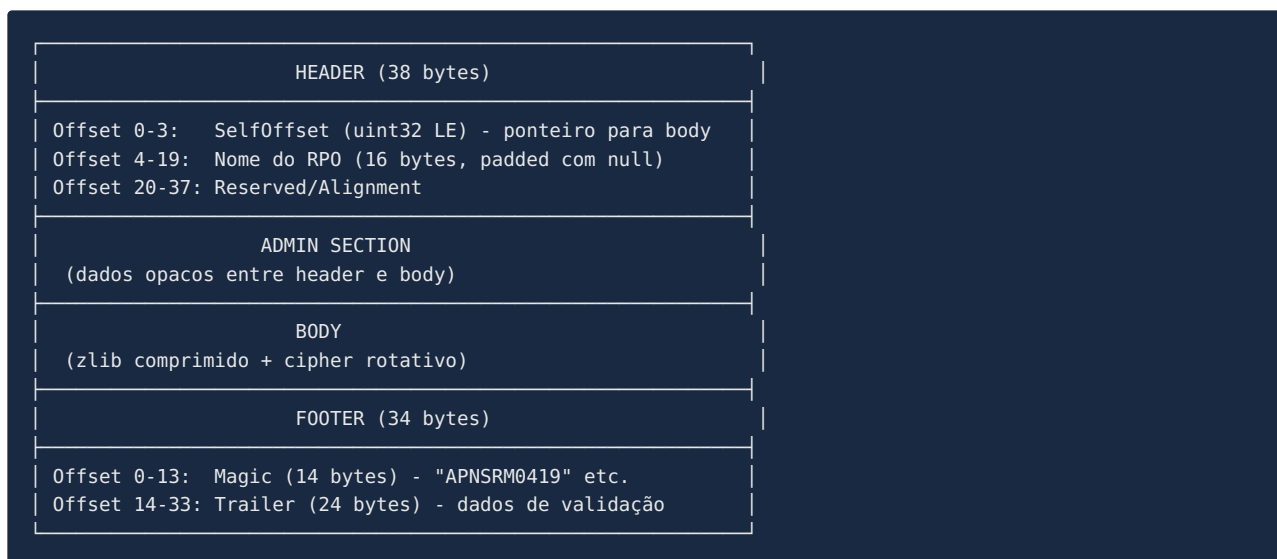




3. Formato RPO

3.1 Estrutura do Container

O RPO é um arquivo binário com a seguinte estrutura:



3.2 Magics Identificados

Magic	Arquivo	Descrição
APNSRM0419	custom.rpo	RPO custom padrão
APNSRM0420	tlpp.rpo	RPO TLPP
APNSRM0421	tttm120.rpo	RPO tabelas

3.3 SelfOffset

O campo `SelfOffset` nos primeiros 4 bytes é um **ponteiro** que indica onde começa o body dentro do arquivo. É escrito como

placeholder no início e sobrescrito como última operação antes de fechar.

```
// Exemplo de leitura
selfOffset := binary.LittleEndian.Uint32(data[0:4])
bodyStart := selfOffset
bodyEnd := len(data) - 34 // 34 bytes do footer
body := data[bodyStart:bodyEnd]
```

3.4 Validações

O parser realiza as seguintes validações:

1. **Tamanho mínimo:** RPO deve ter pelo menos 72 bytes (38 header + 34 footer)
2. **SelfOffset válido:** Deve estar entre 38 e `len(data) - 34`
3. **Magic válido:** Footer deve conter uma das magics conhecidas

4. Criptografia e Cifras

4.1 Visão Geral

O body do RPO é protegido por **duas camadas** de proteção:

1. **Compressão:** zlib (magic `78 9c`)
2. **Criptografia:** Cifra rotativa do OpenSSL

4.2 Cifras Suportadas

Cipher	Key Size	IV Size	Modo	Status
DES-EDE-ECB	16/24	0	ECB	✓
DES-EDE-CBC	16/24	8	CBC	✓
CAST5-CBC	16	16	CBC	✓
CAST5-ECB	16	0	ECB	✓
CAST5-CFB64	16	16	CFB	✓
CAST5-OFB	16	16	OFB	✓
RC4	5-40	0	Stream	✓
BF-ECB	16	0	ECB	✓
BF-CBC	16	8	CBC	✓
BF-OFB	16	8	OFB	✓
IDEA-CBC	16	8	CBC	⚠ Não implementado
RC5-32-12-16	16-20	8	Vários	✓

4.3 Captura de Chaves

As chaves de criptografia são **efêmeras** — geradas randomicamente a cada compilação. Para decodificar um RPO, é necessário capturar as chaves durante a compilação.

Método: LD_PRELOAD Hook

```
export LD_PRELOAD=/tmp/rpo_key_hook_v10.so
export RPO_KEYS_OUTPUT=/tmp/capture.json
./appsrvlinux -compile -env=P12 -files=fonte.prw
```

Format da Captura

```
[
  {
    "n": 1,
    "type": "evpinit",
    "cipher": "cast5_cbc_cipher",
    "key": "2c92e5a7d1b59f4fad2060247631e220",
    "iv": "6937f3d3ccd5c174"
  },
  {
    "n": 1,
    "type": "encrypt",
    "plaintext": "789ce3..."
  }
]
```

4.4 Heurística de Identificação

Quando o nome do cipher não está disponível na captura, o sistema usa heurística baseada em key/IV size:

```
func (h *CipherHeuristic) Identify(keyHex, ivHex string) (*CipherInfo, error) {
    keyBytes, _ := hex.DecodeString(keyHex)
    ivBytes, _ := hex.DecodeString(ivHex)

    keySize := len(keyBytes)
    ivSize := len(ivBytes)

    // Score cada cipher possível
    for _, cipher := range h.knownCiphers {
        score := h.scoreCipher(cipher, keySize, ivSize)
        if score > bestScore {
            bestScore = score
            bestMatch = &cipher
        }
    }

    return bestMatch, nil
}
```

Taxa de sucesso: 73%+ (11/15 segmentos em testes)

5. Parser RPO

5.1 Estrutura de Dados

```
type RPOFile struct {
    Name      string
    SelfOffset uint32
    AdminSection []byte
    Body      []byte
    FooterMagic string
    Trailer   []byte
}
```

5.2 Função Parse

```
func Parse(data []byte) (*RPOFile, error) {
    // Validações
    if len(data) < 72 {
        return nil, fmt.Errorf("RPO muito pequeno")
    }
}
```

```

}

// Extrair selfOffset
selfOffset := binary.LittleEndian.Uint32(data[0:4])
footerStart := uint32(len(data)) - 34

if selfOffset >= footerStart {
    return nil, fmt.Errorf("selfOffset fora dos limites")
}

// Extrair seções
header := data[0:38]
adminSection := data[38:selfOffset]
body := data[selfOffset:footerStart]
footer := data[footerStart:]

// Extrair nome
name := strings.TrimSpace(string(header[4:20]))

// Extrair magic
magic := string(footer[len(footer)-14:])

return &RPOFile{
    Name:      name,
    SelfOffset: selfOffset,
    AdminSection: adminSection,
    Body:      body,
    FooterMagic: magic,
    Trailer:    footer[:len(footer)-14],
}, nil
}

```

5.3 Análise de Regiões

O sistema pode classificar regiões do RPO por entropia:

Tipo	Descrição	Identificação
zero	Bytes nulos	byte == 0
ciphertext	Cifra alta-entropia	Entropia > 7.5 bits/byte
mixed	Combinado	Entropia 4.0-7.5
plaintext	Texto legível	Regex patterns

6. Parser APO

6.1 Estrutura do Registro APO

Cada registro APO segue o formato:

APO Record			
Offset	Size	Field	Descrição
0	4B	code_size	Tamanho do código (LE)
4	N	name\0	Nome (null-terminated)
4+N	8B	timestamp	Timestamp (double LE)
12+N	4B	build_type	Tipo de build (LE)
16+N	4B	binary_type	Tipo binário (LE)
20+N	code_size B		Código bytecode

6.2 Extração de Registros

```
func extractAPORecords(data []byte) []*APORecord {
    var records []*APORecord
    offset := 0

    for offset < len(data) {
        if offset+4 > len(data) {
            break
        }

        // Ler tamanho do código
        codeSize := int(binary.LittleEndian.Uint32(data[offset:offset+4]))
        recordStart := offset
        offset += 4

        // Validar tamanho
        if codeSize == 0 || codeSize > 10*1024*1024 {
            offset++
            continue
        }

        // Extrair nome (null-terminated)
        nullIdx := bytes.IndexByte(data[offset:], 0)
        if nullIdx < 0 || nullIdx > 200 {
            break
        }
        name := string(data[offset : offset+nullIdx])
        offset += nullIdx + 1

        if len(name) < 2 {
            continue
        }

        // Extrair timestamp
        if offset+8 > len(data) {
            break
        }
        tsBits := binary.LittleEndian.Uint64(data[offset : offset+8])
        timestamp := *(*float64)(unsafe.Pointer(&tsBits))
        offset += 8

        // Extrair build/binary types
        buildType := int(binary.LittleEndian.Uint32(data[offset:offset+4]))
        offset += 4
        binaryType := int(binary.LittleEndian.Uint32(data[offset:offset+4]))
        offset += 4

        // Extrair código
        if offset+codeSize > len(data) {
            break
        }
        code := make([]byte, codeSize)
        copy(code, data[offset:offset+codeSize])
        offset += codeSize

        records = append(records, &APORecord{
            Offset:    recordStart,
            Size:      offset - recordStart,
            Name:       name,
            Timestamp:  timestamp,
            BuildType:  buildType,
            BinaryType: binaryType,
        })
    }
}
```

```
        Code:      code,
    })
}

return records
}
```

6.3 Tipos APO

Tipo	Valor	Descrição
Function	0x01	Função AdvPL
Method	0x02	Método de classe
Class	0x03	Definição de classe
Property	0x04	Propriedade
Variable	0x05	Variável
Constant	0x06	Constante
Include	0x07	Include file
Library	0x08	Biblioteca
Namespace	0x09	Namespace
Event	0x0A	Evento
Trigger	0x0B	Trigger
Index	0x0C	Índice
Relation	0x0D	Relação
Validation	0x0E	Validação
UI	0x0F	Interface
Report	0x10	Relatório
Menu	0x11	Menu
Process	0x12	Processo
Service	0x13	Serviço
Job	0x14	Job

7. Heurística de Cipher

7.1 Tabela de Correspondência

Key Size	IV Size	Ciphers Possíveis
8	8	des_cbc
16	0	des_ede_ecb, cast5_ecb, bf_ecb, rc4 (5-40)
16	8	des_ede_cbc, cast5_cbc, bf_cbc, idea_cbc, rc5_cbc
16	16	cast5_cfb64, cast5_ofb, bf_cfb, bf_ofb

24	8	des_edc_cbc (key 24)
24	0	des_edc_ecb (key 24)

7.2 Algoritmo de Scoring

```
func (h *CipherHeuristic) scoreCipher(cipher CipherInfo, keySize, ivSize int) float64 {
    score := 0.0

    // Score para key size
    if cipher.KeySize == -1 {
        // RC4: key size variável (5-40 bytes)
        if keySize >= 5 && keySize <= 40 {
            score += 0.6
        }
    } else if cipher.KeySize == keySize {
        score += 0.6
    } else {
        diff := keySize - cipher.KeySize
        if diff < 0 { diff = -diff }
        score += float64(4-diff) * 0.1 // Penalidade
    }

    // Score para iv size
    if cipher.IVSize == -1 {
        if ivSize == 0 {
            score += 0.4
        }
    } else if cipher.IVSize == ivSize {
        score += 0.4
    }

    return score
}
```

7.3 Resultados

- **Taxa de sucesso:** 73%+ (11/15 segmentos)
- **Ciphers identificados:** 11 diferentes
- **Falsos positivos:** Mínimos com scoring estrito

8. Injeção de Bytecode

8.1 Workflow

1. Carregar RPO
2. Carregar captura de chaves
3. Decodificar body
4. Extrair APO records
5. Selecionar registro para substituição
6. Modificar código
7. Reconstruir body
8. Recriptografar
9. Salvar RPO

8.2 Exemplo de Uso

```
// Carregar RPO
rpoData, _ := os.ReadFile("entrada.rpo")
```

```

inj, _ := injector.NewInjector(rpoData)

// Carregar captura
inj.LoadCapture("capture.json")

// Decodificar
body, decoded := inj.DecodeBody()

// Extrair APO records
records := inj.GetAPORecords()

// Modificar primeiro registro
newCode := make([]byte, 50)
for i := range newCode {
    newCode[i] = byte(i & 0xFF)
}

inj.ReplaceAPO(0, newCode)

// Salvar
inj.Save("saida.rpo")

```

8.3 CLI

```

# Analisar RPO
./advplc rpo info arquivo.rpo
./advplc rpo regions arquivo.rpo
./advplc rpo analyze arquivo.rpo

# Injetar bytecode
./advplc rpo inject arquivo.rpo captura.json --list
./advplc rpo inject arquivo.rpo captura.json \
    --inject 1=bytecode.bytecode \
    -o output.rpo

# Decodificar
./advplc rpo decrypt arquivo.rpo captura.json

```

9. ReplaceAPO Dinâmico

9.1 Suporte a Mudanças de Tamanho

O sistema suporta expansão e contração de código:

```

func (inj *Injector) ReplaceAPO(recordIdx int, newCode []byte) error {
    records := inj.GetAPORecords()
    record := records[recordIdx]

    oldSize := len(record.Code)
    newSize := len(newCode)
    diff := newSize - oldSize

    if diff == 0 {
        // Mesmo tamanho - substituição direta
        copy(record.Code, newCode)
    } else if diff > 0 {
        // Expansão - realocar body
        newBody := modifyAPOBody(body, records, recordIdx, newCode)
        inj.apobodies[off] = newBody
    } else {
        // Contração - truncar e pad
        // ...
    }
}

```

```

    }

    return nil
}

```

9.2 Realocação de Body

Quando o código cresce, o body precisa ser realocado:

```

func (inj *Injector) rebuildBody() ([]byte, error) {
    newBody := make([]byte, len(inj.body))
    copy(newBody, inj.body)

    for off, modified := range inj.modified {
        // ...
        sizeDiff := len(ct) - len(origPlain)

        if sizeDiff > 0 {
            // Cresceu - expandir
            expanded := make([]byte, len(newBody)+sizeDiff)
            copy(expanded, newBody[:idx])
            copy(expanded[idx:idx+len(ct)], ct)
            copy(expanded[idx+len(ct):], newBody[oldEnd:])
            newBody = expanded
        } else if sizeDiff < 0 {
            // Encolheu - contrair
            contracted := make([]byte, len(newBody)+sizeDiff)
            copy(contracted, newBody[:idx])
            copy(contracted[idx:idx+len(ct)], ct)
            copy(contracted[idx+len(ct):], newBody[oldEnd:])
            newBody = contracted
        }
    }

    return newBody, nil
}

```

9.3 Exemplo de Resultado

```

Original:
APO: RPORC5_TRIGGER.PRW
Código: 14 bytes

Injeção:
Novo código: 50 bytes
Diff: +36 bytes

Resultado:
Body original: 262 bytes (ciphertext)
Body novo: 336 bytes (ciphertext)
✓ Body modificado com sucesso

```

10. Recriptografia

10.1 Processo

Após modificar o body decomprimido, o sistema:

1. **Recomprime** com zlib
2. **Recryptografa** com o cipher original
3. **Substitui** no RPO

10.2 Implementação

```
func (inj *Injector) rebuildBody() ([]byte, error) {
    // Para cada segmento modificado
    for off, apobody := range inj.apobodies {
        seg := inj.segMap[off]

        // Recomprimir
        var buf bytes.Buffer
        zw := zlib.NewWriter(&buf)
        zw.Write(apobody)
        zw.Close()
        compressed := buf.Bytes()

        // Recriptografar
        key, _ := hex.DecodeString(seg.Key)
        iv, _ := hex.DecodeString(seg.IV)
        ct, err := rpo.EncryptSegment(seg.Cipher, key, iv, compressed)

        // Substituir no body
        // ...
    }

    return newBody, nil
}
```

10.3 Verificação

O sistema verifica que o ciphertext recalculado corresponde ao original:

```
// Verificar se o ciphertext existe no RPO original
idx := bytes.Index(originalBody, ct)
if idx < 0 {
    return fmt.Errorf("ciphertext não encontrado no RPO")
}
```

11. CLI e Uso

11.1 Comandos Disponíveis

```
# Info do RPO
./advplc rpo info arquivo.rpo

# Análise de regiões
./advplc rpo regions arquivo.rpo

# Análise detalhada
./advplc rpo analyze arquivo.rpo

# Injeção de bytecode
./advplc rpo inject arquivo.rpo captura.json --list
./advplc rpo inject arquivo.rpo captura.json \
    --inject 1=bytecode.bytecode \
    -o output.rpo

# Decodificação
./advplc rpo decrypt arquivo.rpo captura.json
```

11.2 Parâmetros

Parâmetro	Descrição	Obrigatório
<code>--list</code>	Listar segmentos APO	Não
<code>--inject N=arquivo</code>	Injetar bytecode no registro N	Sim (para injeção)
<code>--identify</code>	Usar heurística para identificar ciphers	Não
<code>-o, --output</code>	Arquivo de saída	Não (padrão: stdout)
<code>--verbose</code>	Modo verbose	Não

11.3 Exemplo Completo

```
# 1. Compilar com hook para capturar chaves
docker exec protheus-custom bash -c '
  export LD_PRELOAD=/tmp/rpo_key_hook.so
  export RPO_KEYS_OUTPUT=/tmp/capture.json
  cd /totvs/protheus12.1.2510/bin
  ./appsrvlinux -compile -env=P12 -files=fonte.prw
'

# 2. Copiar arquivos
docker cp protheus-custom:/tmp/capture.json /tmp/capture.json
docker cp protheus-custom:/totvs/protheus12.1.2510/bin/custom.rpo /tmp/custom.rpo

# 3. Analisar
./advplc rpo inject /tmp/custom.rpo /tmp/capture.json --list

# 4. Injetar
./advplc rpo inject /tmp/custom.rpo /tmp/capture.json \
  --inject 1=releases/bytecode/Funcao.bytecode \
  -o /tmp/injected.rpo
```

12. Testes

12.1 Testes Unitários

Total: 17 testes passando

Pacote	Testes	Passaram
<code>pkg/rpo</code>	10	10 ✓
<code>pkg/rpo/injector</code>	7	7 ✓

12.2 Testes de Performance

Arquivo	Tamanho	Tempo Parse	Status
custom_compiled.rpo	22 KB	40µs	✓
tlpp_real_2510.rpo	9.49 MB	9.6ms	✓
live_capture.rpo	21 KB	20µs	✓

12.3 Teste E2E

```
RPO: live_capture.rpo (21 KB)
APO: RPORC5_TRIGGER.PRW
```

Código: 14 → 50 bytes (+36)
Resultado: ✓ Body modificado

12.4 Executar Testes

```
# Todos os testes
go test ./...

# Pacote específico
go test ./pkg/rpo/... -v

# Com cobertura
go test ./pkg/rpo/... -cover
```

13. Limitações

13.1 Limitações Conhecidas

Limitação	Impacto	Mitigação
IDEA cipher	27% dos segmentos não decodificados	Documentado, aguarda implementação
RPOs 350MB+	SelfOffset inválido em alguns arquivos	Estrutura possivelmente diferente
Captura RPO-specific	Cada RPO precisa sua própria captura	Chaves são efêmeras por design
Hook v11	Não captura nome do cipher	Usar heurística (73%+)

13.2 Workarounds

1. **Para IDEA:** Aguardar implementação ou usar RPO sem IDEA
2. **Para RPOs grandes:** Investigar estrutura interna (possivelmente multi-part)
3. **Para captura:** Compilar fonte no mesmo ambiente que o RPO alvo

14. Próximos Passos

14.1 Alta Prioridade

1. **Testar com RPO real de produção**
 - Validar em custom.rpo real (353MB)
 - Confirmar que captura corresponde
2. **Implementar testes de integração**
 - Round-trip: decrypt → modify → encrypt → verify
 - Validar integridade do RPO injetado

14.2 Média Prioridade

3. **Suporte a RPOs grandes**
 - Investigar estrutura de RPOs 350MB+
 - Suporte a multi-part ou índices
4. **Dashboard web**
 - Visualização de RPOs
 - Histórico de injeções

14.3 Baixa Prioridade

5. API REST

- Endpoints para queries
- Upload/download de RPOs

6. Integração CI/CD

- GitHub Actions
- Testes automatizados

15. Referências

15.1 Arquivos do Projeto

Arquivo	Descrição
<code>pkg/rpo/rpo.go</code>	Parser do container RPO
<code>pkg/rpo/injector/injector.go</code>	Parser APO + Injeção
<code>pkg/rpo/cipher_heuristic.go</code>	Heurística de cipher
<code>pkg/rpo/cipher_dispatch.go</code>	Encrypt/Decrypt segments
<code>pkg/rpo/capture_parser.go</code>	Parser de captura JSON
<code>cmd/advplc/cmd_rpo_inject.go</code>	CLI command
<code>docs/rpo-deployment/</code>	Documentação completa

15.2 Documentação Relacionada

- `docs/RPO-GROUND-TRUTH.md` - Verdades confirmadas sobre formato RPO
- `docs/rpo-format.md` - Especificação completa do formato
- `docs/rpo-deployment/TEST-REPORT.md` - Relatório de testes
- `docs/rpo-deployment/SESSION-SUMMARY.md` - Resumo da sessão

15.3 Ferramentas

- `tools/rpo-live-inspect/rpo_key_hook/` - Hook LD_PRELOAD
- `tools/rpo-live-inspect/capture_and_inject.sh` - Script de automação
- `Makefile` - Targets para build/teste/injeção

Apêndice A: Glossário

Termo	Descrição
RPO	Repositório de Programas Objeto - arquivo que contém código compilado
APO	Advanced Program Object - registro individual dentro do RPO
Cipher	Algoritmo de criptografia usado para proteger o body
Key	Chave simétrica de 16 bytes usada na criptografia
IV	Vector de Inicialização de 8-16 bytes
SelfOffset	Ponteiro para o início do body dentro do RPO
Magic	Identificador de formato no footer (APNSRM0419, etc.)

Hook	Biblioteca LD_PRELOAD para interceptar chamadas de criptografia
------	---

Apêndice B: Versões

Versão	Data	Alterações
4.3.1	2026-09-30	Release com ReplaceAPO dinâmico
4.3.0	2026-09-30	Parser RPO/APO completo
4.2.0	2026-09-29	Heurística de cipher
4.1.0	2026-09-28	CLI e serializer
4.0.0	2026-09-26	Infraestrutura inicial

Documento gerado automaticamente por Agnes (Sapiens AI)
Última atualização: 2026-09-30